Requirements Analysis and Design

COMP3095 – Capstone Project 1
QuietSpace

Group 3

Team Members:
Baris Isci – 101531238
Daniil Orlov - 101500729
Ibrahim Yondem - 101525165
Gozde Baran - 101515982
Nguyen Minh Triet Luu – 101542519

Instructor:
Anjana Shah

Table of Contents

1.0 INTRODUCTION *4*

1.1 Purpose **4**

1.2 Scope **4**

1.3 Objectives and benefits **5**

2.0 SYSTEM OVERVIEW *5*

2.1 Project Perspective **5**

2.2 System Context **6**

2.2.1 Business Case and Vision 6

2.2.2 Strategic Positioning and Market Fit 7

2.3 General Constraints **7**

2.3.1 Platform Technology: Cross-Platform Mobile Application 7

2.3.2 Backend and Authentication Service: Third-Party BaaS Provider 8

2.3.3 Deployment Target: Public App Stores 9

2.3.4 Project Timeline: End-of-Semester Deadline 9

2.3.5 Target Audience: George Brown College Students 9

2.4 Assumptions and Dependencies **10**

2.4.1 Assumptions: 10

2.4.2 Dependencies: 11

3.0 FUNCTIONAL REQUIREMENTS *12*

3.1 Functional Requirement or Feature **12**

FR1: Map-Based Quiet Location Discovery 12

FR2: User-Contributed Location Submission 13

FR3: Real-Time Check-In and Condition Updates 13

FR4: User Accounts and Profile Management 14

FR5: Gamification – Points and Badge System 15

FR6: Favorites and Bookmarking Locations 15

FR7: Location Search and Filtering 16

FR8: Built-In Pomodoro Timer (Productivity Tool - Optional) 17

FR9: Location Details, Reviews and Ratings 17

FR10: Social and Community Features (Friends & Posts) 18

FR11: Customization and Settings 19

3.2 Use Cases **20**

3.3 Data Modelling and Analysis **30**

3.4 Process Modelling **37**

4.0 NON-FUNCTIONAL REQUIREMENT *37*

4.1 Performance **37**

4.2 Reliability and Availability **38**

4.3 Security and Privacy **38**

4.4 Maintainability and Scalability **38**

4.5 Portability and Compatibility **38**

4.6 Usability and Accessibility **38**

5.0 LOGICAL DATABASE REQUIREMENT *39*
5.1 Data Formats **39**
5.2 Storage Capabilities **39**
5.3 Data Retention **39**
5.4 Data Integrity **40**
5.5 Security and Access **40**
5.6 Backup and Recovery **40**
6.0 OTHER REQUIREMENTS *40*
7.0 APPROVAL *40*

1.0 INTRODUCTION

QuietSpace is a mobile application that helps students, remote workers, and freelancers find reliable, quiet places to focus. The app presents a community driven map of cafés, libraries, campus rooms, and coworking spots. Each location shows practical details such as current noise level, Wi-Fi quality, seating availability, power outlets, and recent verification, with optional social features that let users see friends or plan sessions together. Users can check in to report real time conditions, save favorites, apply smart filters, and use productivity tools like a Pomodoro timer. The project addresses a documented problem where many learners and knowledge workers spend significant time searching for a suitable place and often arrive to find poor Wi-Fi or limited seating. QuietSpace reduces this waste with crowd-verified data, a trust system, and helpful tools for focus.

1.1 Purpose

A mobile application that helps students, remote workers, and freelancers discover, evaluate, and choose reliable quiet places to study or work by providing up-to-date, crowd-sourced condition reports (noise level, seating availability, Wi-Fi reliability, outlet access, lighting), location discovery via interactive maps and smart filters, real-time check-ins and freshness indicators, privacy-first account controls, and lightweight productivity tools so users can confidently find and navigate to the best nearby spaces while minimizing time wasted searching.

1.2 Scope

Native Android application. Interactive map with location submission and management. Real time check-ins that capture noise, seating, Wi-Fi, and outlets using simple controls. User accounts with privacy controls. Ratings and reviews. Favorites. Smart filtering and sorting by quietest, highest rated, most recently updated, or study mode compatibility. Gamification with badges and contribution points. Optional social visibility to see friends or share location with study partners. Built in Pomodoro timer with customizable presets. Trust signals that show when a place was last verified and by how many users. Cloud backend and database for storage, synchronization, and authentication.

1.3 Objectives and Benefits

Reduce time spent searching for study or work locations. Improve focus by matching users with spaces that fit noise and Wi-Fi preferences. Build a sustainable community that keeps information fresh through check-ins. Create a scalable foundation that can later expand to partnerships and smarter recommendations.

2.0 SYSTEM OVERVIEW

The System Overview section introduces the system context and design.

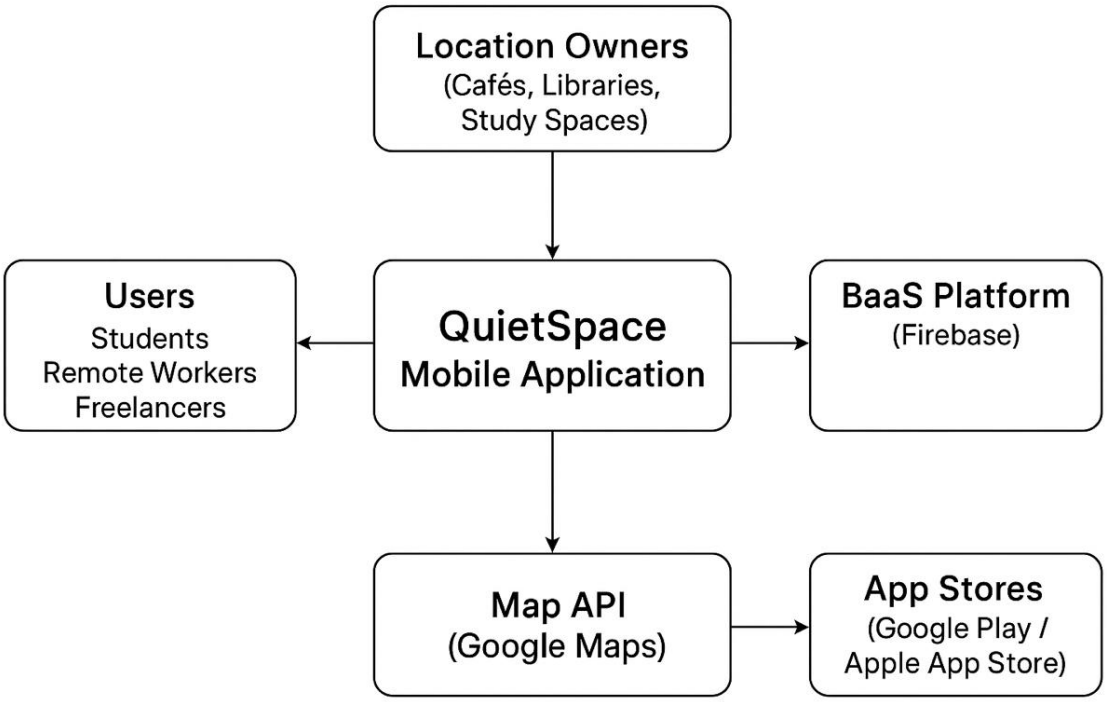
2.1 Project Perspective

The QuietSpace system is categorized as a **new self-contained system**, not a follow-on or replacement for an existing campus application. This means the project must establish its value proposition and user base from the ground up, facing challenges associated with building a **"New Brand"** and overcoming a **"Slow Start"** (the "empty room" problem of limited initial content).

- **System Origin and Scope:** QuietSpace is envisioned as a **mobile application** designed to help students, remote workers, and freelancers discover reliable, quiet places to study or work outside their homes. It aims to build a trusted network of spaces that support focus, connection, and long-term growth.
- **Expansion Trajectory:** While starting as a self-contained mobile app focused on core map and check-in features, the perspective includes a clear path for future expansion. Later enhancements include a desktop web application, advanced "Study Buddy" social features, potential integration with university systems, and even AI-powered space availability predictions.

2.2 System Context

The System Context describes the problem QuietSpace solves and the strategic position it aims to occupy within the evolving remote work and academic environment.



System Context Diagram

2.2.1 Business Case and Vision

The core vision is to make **productivity accessible anywhere**. The business justification stems from the rapidly **growing demand for quiet, reliable study and workspaces**, driven by a surge in remote work and hybrid learning.

The application addresses a gap left by general mapping services by offering specialized, community-vetted information:

- **Live, User-Generated Data:** QuietSpace differentiates itself by using **live data from users** to guide others to the best nearby locations. This directly addresses the need for real-time status updates on factors like noise level and busyness, which traditional directories lack.

- **Targeted Value Proposition:** It provides specific value to its four key user groups—**Students, Remote Workers, Freelancers, and Location Owners** —by offering a reliable source of information for environments conducive to concentration.

2.2.2 Strategic Positioning and Market Fit

The project is strategically positioned to capitalize on opportunities while mitigating key risks in the market:

- **Market Opportunity:** The project identifies **University & Campus Partnerships** and **Business Collaborations**(like cafés and coworking spaces) as major opportunities. These partnerships would provide fast access to target users, increase credibility, and attract more venues, making the app a strong fit for the expanding market.

- **Mitigation of Competition:** While facing strong competition from platforms like Google Maps and Yelp , QuietSpace plans to use **Gamification** to reward participation and promote engagement , thereby building a defensible asset of high-quality, community-driven data that competitors can't easily replicate.

- **Core Strategic Risk:** The project faces the **High Risk** of low user engagement, also known as the "empty room" problem. The system context inherently relies on a critical mass of users to **actively and consistently provide the ratings and reviews** that form the app's core value. Mitigation strategies, such as gamification and team pre-population of initial data, are considered essential for long-term success.

2.3 General Constraints

This section provides a granular analysis of the absolute, unchangeable rules that define the project's boundaries.

2.3.1 Platform Technology: Cross-Platform Mobile Application

- **Detail:** The project mandates the creation of a single application codebase that can be compiled and deployed to both Apple's iOS and Google's Android operating systems. This will be achieved using a cross-platform framework such as React Native or Flutter. This choice explicitly forbids developing two separate, native applications (in Swift/Kotlin) or a progressive web app (PWA).

- **Technical Impact:** This constraint is the most significant driver of technology selection.
- **Performance:** The application's performance will be subject to the overhead of the chosen framework's abstraction layer (e.g., the JavaScript bridge in React Native). While generally sufficient for this type of application, it will not be as performant as a true native app.
- **UI/UX:** The user interface must be designed to function effectively on both platforms, which have different design philosophies (e.g., Apple's Human Interface Guidelines vs. Google's Material Design). This can lead to a more generic UI or require significant effort to create platform-specific adaptations.
- **Hardware Access:** Access to native device features (like advanced camera functions or specific sensors) is limited to what the framework's APIs expose, which can be a step behind native capabilities.
- **Project Management Impact:** The development team must possess or acquire skills in the chosen cross-platform framework. The testing and quality assurance process is more complex, as every feature must be verified on a range of iOS and Android devices to check for platform-specific bugs.

2.3.2 Backend and Authentication Service: Third-Party BaaS Provider

- **Detail:** The project is architecturally constrained to a client-server model where the entire backend is provided by a third-party Backend-as-a-Service (BaaS) platform (e.g., AWS Amplify, Google Firebase, Supabase). This includes the database, user authentication, and server-side logic.
- **Technical Impact:**
- **Vendor Lock-In:** This creates a strong dependency on the chosen provider. Migrating to a different backend in the future would be a massive undertaking, as it would require rewriting the entire data access layer and authentication system.
- **Limited Customization:** The team is limited to the features, database types, and server-side logic capabilities offered by the BaaS provider. Complex, custom backend processes might be difficult or impossible to implement.
- **Data Management:** All user data will be stored on the provider's servers, which has significant data privacy and security implications. The team must consider the provider's compliance with local data protection regulations (like PIPEDA in Canada).
- **Project Management Impact:** This dramatically reduces the backend development workload, allowing the team to focus almost exclusively on the client-side mobile application. However, it introduces a new operational cost and requires the team to manage API keys and service configurations securely.

2.3.3 Deployment Target: Public App Stores

- **Detail:** The final deliverable must be a complete, signed, and packaged application ready for submission to the public Apple App Store and Google Play Store.
- **Technical Impact:** The application must be built to a production standard, with robust error handling and performance optimization. It must also include all necessary metadata, such as various icon sizes, feature graphics, and a clearly written privacy policy. For iOS, this involves managing complex certificates, identifiers, and provisioning profiles.
- **Project Management Impact:** This adds several non-coding tasks to the project plan, including creating store listings and navigating the potentially lengthy and opaque app store review processes. It also introduces a direct financial cost, specifically the annual fee for the Apple Developer Program. There is a risk of rejection, which could delay the project's completion.

2.3.4 Project Timeline: End-of-Semester Deadline

- **Detail:** This is a fixed-duration constraint of approximately 12-14 weeks. It is a non-negotiable deadline that dictates the project's scope and pace.
- **Technical Impact:** This constraint forces the team to prioritize simplicity and reuse. It increases the risk of accumulating "technical debt"—making short-term design choices for the sake of speed that may make future development more difficult. Complex or time-consuming features must be ruthlessly cut.
- **Project Management Impact:** This necessitates a rigorous project management methodology, such as Scrum with short, focused sprints. The scope of the Minimum Viable Product (MVP) must be defined within the first 1-2 weeks and then strictly enforced. The timeline must account for all phases, including design, development, testing, and the app store submission process.

2.3.5 Target Audience: George Brown College Students

- **Detail:** The application is not for a general audience; it is a niche product designed to serve the specific needs of students at a particular institution.
- **Technical Impact:** This may require integration with college-specific systems (though this project uses a general BaaS) or branding. The location data and map overlays must be specific to GBC campuses.
- **Project Management Impact:** This provides a clear, accessible group for user research, requirements gathering, and User Acceptance Testing (UAT). The project's success will be measured by its adoption and utility within this specific community, so user feedback from GBC students is a critical asset.

2.4 Assumptions and Dependencies

This section provides a risk-aware analysis of the project's foundational beliefs and external requirements.

2.4.1 Assumptions:

User Technology Access and Proficiency: We assume the vast majority of GBC students own a smartphone running a relatively recent version of iOS or Android and are comfortable downloading and using new apps.

- **Nuance:** This assumption overlooks students with older, less powerful devices, limited mobile data plans, or accessibility needs.
- **Risk:** Low, but with notable implications. To mitigate the impact, the app should be tested on older hardware, be mindful of its data consumption, and be designed following Web Content Accessibility Guidelines (WCAG) to be usable by students with disabilities.

User Engagement and Content Contribution: We assume that a critical mass of users will actively and consistently provide the ratings and reviews that form the core value of the application.

- **Nuance:** This assumes users will overcome the "empty room" problem (an app with no data is not useful). It also assumes the data they provide will be of reasonable quality.
- **Risk:** High. This is the primary existential risk to the project's long-term value. Mitigation strategies are not optional; they are essential. This includes gamification (badges, leaderboards), push notification reminders, making the submission process as frictionless as possible, and having the project team pre-populate the app with a substantial amount of initial data before launch.

Sufficiency of BaaS Provider's Free Tier: We assume the free service tier of the chosen BaaS provider will be sufficient for all development, testing, and final presentation activities.

- **Nuance:** Free tiers have hard limits on metrics like concurrent users, API calls per day, and total data storage. A successful launch or even a well-attended presentation could exceed these limits.
- **Risk:** Medium. Exceeding these limits would result in a temporary but critical service outage. Mitigation requires designating a team member to monitor the BaaS provider's usage dashboard, understanding the pricing model in case a temporary upgrade is needed, and load-testing the application before the final presentation.

2.4.2 Dependencies:

Critical Reliance on the BaaS Platform: The project is wholly dependent on the chosen third-party BaaS provider for all backend functionality.

- **Nuance:** This means the project's uptime, performance, and security are largely in the hands of a third party. It is a single point of failure.
- **Risk:** Medium. A service outage at the provider means our application is completely non-functional. Vendor lock-in is a major strategic risk; if the provider changes its pricing or discontinues a critical feature, the project has little recourse. Mitigation involves choosing a well-established provider and subscribing to their status page for outage notifications.

Availability and Quality of GBC Campus Data: The application is dependent on obtaining accurate, machine-readable maps and location data for all relevant George Brown College buildings and study areas.

- **Nuance:** This data needs to be more than just a PDF map. Ideally, it would be in a structured format like GeoJSON or at least a list of addresses and room numbers that can be geocoded.
- **Risk:** High. This data may not exist, may be inaccurate, or the college administration may be unwilling or slow to provide it. This is a major blocker. The primary mitigation is to address this dependency in the first week of the project. The backup plan (manually creating this data) must be considered a significant project in itself and be factored into the timeline.

Access to a Specific and Consistent Development Environment: The project is dependent on all team members having access to the necessary hardware and software.

- **Nuance:** This is not just about having a computer. It's about having a **macOS machine** for building the iOS app, the correct versions of Node.js, Xcode, Android Studio, and all associated SDKs. Inconsistent environments across a team can lead to "works on my machine" issues.
- **Risk:** Medium. This is a logistical bottleneck. Mitigation involves conducting a hardware/software audit of the team's resources on day one. The team should use tools like nvm or Docker to ensure that the development environment is as consistent as possible for all members. If a Mac is not available, the team must immediately investigate cloud-based macOS build services.

3.0 FUNCTIONAL REQUIREMENTS

QuietSpace is a mobile application designed to help students, freelancers, and remote workers find and share quiet study/work locations. The app provides an interactive map of community-submitted quiet spots (like libraries, cafés, or campus areas), with details on noise levels, Wi-Fi quality, seating availability, and other conditions. Users can create personal profiles, contribute new locations or real-time updates, earn points and badges for engagement, and utilize productivity tools. This section details the functional requirements of the QuietSpace app and outlines key use cases for its current capabilities (as of Sprint 2).

3.1 Functional Requirement or Feature

Below are the main functional requirements of the QuietSpace application. Each requirement is described with its purpose, the inputs involved, how the system processes those inputs, and the resulting outputs or outcomes.

FR1: Map-Based Quiet Location Discovery

- **Introduction:** The system shall provide an **interactive map interface** that displays quiet locations submitted by the community. This feature is the core of QuietSpace, allowing users to discover nearby study or work spots easily on a map view. Locations are marked with icons or pins and include key details to help users decide where to go.
- **Inputs:** User's current location (via GPS) and optionally user search queries or filter criteria (e.g., specifying the type of venue or required features). No manual input is needed for basic browsing; the map initially centers on the user's location and shows nearby quiet places.
- **Processing:** The app retrieves a list of quiet locations from the database (or an external API) and plots them on the map. This includes processing the user's location to determine which spots are nearby, and applying any search/filter parameters to the location dataset. When the user pans or zooms the map, the system dynamically loads and displays locations in the new map area. Tapping on a location pin triggers the app to fetch detailed information for that spot.
- **Outputs:** A map UI on the mobile screen showing pins for quiet locations around the user. Each pin or marker can be selected to display a short **location preview** (name, rating, noise level, etc.). The output also includes a list or carousel of nearby locations (for example, shown at the bottom of the screen) corresponding to the map view. Selecting a location yields a detailed view with full information (see FR9).

FR2: User-Contributed Location Submission

- **Introduction:** The app shall allow users to **add new quiet locations** to the map. This crowdsourced location submission feature enables café owners, students, or any user to contribute a favorite quiet spot into the system so others can discover it. Each submitted location includes descriptive tags and details.
- **Inputs:** The contributor provides details for the new location: name of the place, address or map pinpoint, and characteristics such as noise level, Wi-Fi quality, number of power outlets, seating capacity, business hours, etc. The user may also upload a photo of the location and select category tags (e.g. *library*, *café*, *outdoor*). *(If the user is the owner of the establishment, they might also mark it as "verified" or similar.)*
- **Processing:** Upon submission, the app validates the input (ensuring required fields are filled, the location coordinates are valid, etc.). The new location data is then saved to the backend database. The system might cross-check if a similar location already exists to avoid duplicates. Once stored, the new location becomes part of the dataset of quiet spaces. In the background, the app could also update any search index or map clustering so that the new spot will appear for other users.
- **Outputs:** A confirmation to the user that the location was successfully added (e.g. a message like "Location added successfully"). The newly added spot now appears on the map interface for all users, marked with the provided details. Other users will see this location pin and can view its information just like other locations. The contributor may also earn **contribution points or badges** for adding a new location (see FR5).

FR3: Real-Time Check-In and Condition Updates

- **Introduction:** QuietSpace provides a **real-time check-in system** allowing users to report current conditions of a location. When users arrive at a quiet spot, they can "check in" to update noise levels, crowding, Wi-Fi speed, etc. This keeps information timely and helps others know what to expect.
- **Inputs:** The primary input is the user's action to check in at a specific location (selected from the map or a list). The user can input the current conditions through a quick survey or sliders, for example: current noise level (e.g. *quiet*, *moderate*, *loud*), Wi-Fi quality (e.g. *strong* or *weak* signal), availability of seating (e.g. *many seats available* or *full*), and any other relevant status (such as *temperature* or *lighting* if applicable). The user may also input a short comment about the environment.
- **Processing:** The app records the check-in by time-stamping it and associating it with the selected location and user. Any numerical ratings or status indicators are aggregated with previous data – for instance, updating the average noise level or marking the location as "recently updated." The system may also push this update to

other users currently viewing the location. If the user has enabled location services, the app could verify that the user is physically at or near the location for authenticity of the check-in.

- **Outputs:** The updated location details are immediately visible to all users. For example, if someone checks in and reports *"quiet atmosphere, plenty of free seats"*, the location's info card and detail page will reflect this (showing recent noise level as *Quiet*, seating status as *Available*, updated just now by a user). The app may visually highlight locations with recent check-ins (e.g. a special icon or text like "Just updated"). The user performing the check-in might receive positive feedback such as an increase in their contribution score. This feature ensures **live updates on noise, Wi-Fi, and seating availability** are available for each location, keeping information fresh for everyone.

FR4: User Accounts and Profile Management

- **Introduction:** The system shall provide **user account creation and profile management** features. Users need to be able to create an account (with options for email/password or third-party logins like Google/Apple), and manage their profile information. Having accounts allows QuietSpace to personalize the experience (saving favorites, tracking contributions, etc.) and fosters trust in contributions.
- **Inputs:** For account registration, the user inputs information such as name, email, and password (or chooses a third-party sign-in method). For profile management, inputs include profile details like a profile photo/avatar upload, a short bio, and preferences (for example, notification settings or privacy settings regarding what information to share with others). When editing the profile, the user provides updated values for these fields.
- **Processing:** During registration, the app validates the inputs (e.g. checks email format and password strength) and then communicates with the authentication service (such as Supabase Auth or similar) to create a new user record. If third-party OAuth is used, the app redirects to the provider and handles the token exchange. Once created, a new profile entry (with default values for stats like contribution count) is stored in the database. For profile updates, the system verifies the user's identity (the user must be logged in) and then writes the changes to the user's profile in the database. Privacy settings processing might involve toggling whether certain profile fields (like full name or email) are visible to others.
- **Outputs:** Feedback to the user on success or failure of the action. On successful registration, the user is logged in and taken to the main app interface (with perhaps a welcome message). A profile page is created for the user, showing their display name, avatar, bio, and statistics (e.g. number of locations added, reviews written, etc.). For profile edits, the outputs are the updated profile information immediately reflected in the profile screen. The system also enforces these settings elsewhere

(for example, if a user set their profile to private, their name or stats may be hidden from others' view in community sections). Overall, this requirement enables a personalized and secure user experience.

FR5: Gamification – Points and Badge System

- **Introduction:** QuietSpace incorporates a **gamified reward system** to encourage user engagement. Users earn **points** for contributions (like adding a new location, writing a review, or checking in), and can achieve **badges** or levels when reaching certain milestones. This requirement adds a fun, motivational layer to the app, increasing community participation.
- **Inputs:** User actions serve as inputs to this system. For example, when a user adds a location, writes a review, checks in, or performs other contributory activities, these events are logged. There is no direct user input form for "earning a badge" – instead, the app automatically tracks contributions. In some cases, the user might view their progress (inputting a request by navigating to their profile or a badges screen).
- **Processing:** The app maintains a **user score** or point tally for each user in their profile data. Each type of contribution has an associated point value (e.g. add a new location = 50 points, successful check-in = 10 points, review = 20 points, etc.). When an action is completed, the system increments the user's points accordingly and checks if any badge thresholds are crossed. Badges are defined (for example: *"Pioneer" badge for first location added,* "Explorer Level 1" for visiting 5 new places, *"Top Reviewer" for 10 reviews, etc.). If the user's new point total or action meets a badge criterion, the system awards that badge to the user's profile. This may involve updating a list of earned badges in the database and possibly notifying the user.
- **Outputs:** The user's profile is updated with their current points and any new badges earned. Outputs visible to the user include a **progress indicator** (like points count or a level bar on their profile) and a collection of badge icons on their profile. When a badge is earned, the app might show a pop-up notification or message (e.g. "Congratulations, you earned the **Community Contributor** badge!"). These badges and points are also partly visible to others in the community – for instance, when viewing another user's profile or contributions, one might see that user's badge icons or a status (this helps establish credibility and reputation within the app's community).

FR6: Favorites and Bookmarking Locations

- **Introduction:** Users can **save favorite locations** in QuietSpace for quick access. This requirement lets a user bookmark a quiet place they like, so they can easily find it later without searching. It enhances user convenience by building a personalized list of preferred study/work spots.

- **Inputs:** The primary input is the user's action to mark (or unmark) a location as a favorite. This is typically a tap on a "favorite" icon (heart or star symbol) on a location's detail page or listing. No textual input is required from the user aside from the tap. The user must be logged in, since favorites are tied to their account.
- **Processing:** When the user taps the favorite icon for a given location, the app toggles that location's favorite status for the user. The system updates the user's profile or a favorites table in the backend to record this relationship (user X has favorited location Y). If it's a new favorite, the record is added; if the user is removing it, the record is deleted. The processing might also fetch an updated list of the user's favorites to ensure the UI is current.
- **Outputs:** The immediate UI feedback is a change in the icon state (for example, the heart icon becomes filled to indicate it's saved). The location now appears in the user's **Favorites list** (usually accessible via a "Favorites" section or screen in the app). The Favorites screen displays all locations the user has bookmarked, typically with basic info and maybe sorted by proximity or alphabetically. From this list, the user can tap a favorite to see its details or to navigate to it on the map. Essentially, the output is a personalized subset of locations for the user, persisted between sessions (since it's stored to their account).

FR7: Location Search and Filtering

- **Introduction:** QuietSpace provides advanced **search and filter** functionality to help users find the most suitable quiet space for their needs. Users can search for locations by name or apply filters (e.g. quietest places, open late, has Wi-Fi, etc.). This requirement ensures users can efficiently narrow down locations based on specific criteria.
- **Inputs:** User-entered search queries (text input for keywords or names of places). Also, filter selections serve as input: for instance, toggling filters such as "Open Now", "Noise Level = Very Quiet", "Has Outlets = Yes", "Wi-Fi = Excellent", or choosing a distance radius. The user interacts with a search bar and filter options (like checkboxes or sliders) in the UI.
- **Processing:** The app processes the search query by querying the location database or in-memory list for matches (comparing the query text against location names, descriptions, or tags). For filters, the system applies the selected criteria to the dataset: for example, it will **filter out** any locations that don't meet the selected conditions (e.g. if the user filters for *"open late"*, only locations whose hours extend beyond the current time are included). The combination of a text query with filters is also handled (e.g. "library" in search plus "Wi-Fi: strong" filter). The processing involves updating the map display and any list of results in real-time as the user refines the search or filter options.

- **Outputs:** A refined set of locations shown to the user. In the UI, the map updates to show only pins that match the search/filters, and/or a list of matching results is displayed. If no locations match, the app outputs a “no results found” message. If filters are applied, some UI indication (like chips or tags) shows which filters are currently active. The outcome is that the user can clearly see the locations that fit their criteria. For example, a user searching after 8 PM with filter “Open Now” will see only venues that are currently open, or a user filtering by “quietest” might see only locations with noise level rated as very quiet. This feature improves the efficiency of finding an ideal spot.

FR8: Built-In Pomodoro Timer (Productivity Tool - Optional)

- **Introduction:** QuietSpace includes a **Pomodoro timer** as a built-in productivity tool. This allows users to engage in focused work sessions and breaks without needing a separate app. The presence of this timer in QuietSpace aligns with the app’s mission to not only help find a location but also to help users stay productive when they get there.
- **Inputs:** The user starts the Pomodoro timer by inputting a command (like pressing a “Start Timer” button). They may have adjustable settings as input as well – for example, setting the desired duration of work and break intervals (e.g. 25 minutes focus, 5 minutes break, or custom lengths). The user might also input whether they want a single session or repeated cycles.
- **Processing:** Once started, the app runs a countdown for the work period. Internally, it schedules an alert or notification for when the timer hits zero. If multiple cycles are chosen, the system will alternate between work and break countdowns as per the Pomodoro technique. The processing continues in the background if the user navigates away (potentially using notifications). The app might also track how many cycles have been completed in a session for user reference.
- **Outputs:** The primary output is a visual timer interface counting down minutes and seconds. When a work interval completes, the app outputs a notification or sound indicating it’s time for a break (and similarly indicates when break time is over). The UI might change to show “Break Time!” vs “Work Time” during transitions. If the app has a productivity stats feature, it could also output a summary such as “You completed 3 focus sessions today.” For the user, this feature outputs an increased sense of structure and achievement, integrated seamlessly while they are at a quiet location.

FR9: Location Details, Reviews and Ratings

- **Introduction:** QuietSpace offers a **community rating and review system** for each location. Users can rate a location (e.g. giving a 5-star rating or a simple like/dislike) and write a short review or comment about their experience. This requirement provides qualitative insights on locations and helps surface the best quiet spots.

- **Inputs:** The user's rating input (could be selecting 1–5 stars or a thumbs-up) and an optional text review. To add a review, the user navigates to a location's detail page and fills out a "Write a Review" form which includes a text box and the rating control. Other inputs include viewing reviews – when a user opens a location page, the app fetches existing reviews for display (no direct input from the viewing user except the selection of the location).
- **Processing:** When a new rating/review is submitted, the app validates the content (e.g. ensuring a text review isn't empty if a comment is required, and that it meets community guidelines). It then saves the review in the database linked to that location and the user's profile. The system may also update the location's aggregate rating (for example, calculate a new average star rating). In addition, the processing might involve moderation flags (if certain keywords are banned or if users can report inappropriate reviews, though that's beyond basic scope). On the retrieval side, when displaying a location's details, the app fetches all reviews for that location, sorted by date or usefulness.
- **Outputs:** On the location's detail screen, users see an **aggregate rating** (e.g. an average star rating or score) and a list of user reviews. Each review shows the reviewer's name (or alias), their rating, and their comment. For example, a location might show "Average Rating: 4.5/5 ☆ based on 10 reviews" and then a scrollable list of comments like *"Great quiet spot with reliable Wi-Fi – stayed productive for hours!"*. After a user submits their own review, they receive confirmation (and their review instantly appears in the list). Their contribution will also increment their review count on their profile. This system output provides a richer description of each location through community feedback.

FR10: Social and Community Features (Friends & Posts)

- **Introduction:** QuietSpace includes optional **social connectivity** features to build a sense of community among users. This involves allowing users to connect with or "follow" each other, see when friends are nearby or have visited locations, and engage in a community feed. A built-in **community posts** feature lets users share tips, ask questions, or coordinate study sessions.
- **Inputs:** For friend connections, input is the user's action to add or follow another user (for example, searching for a username and clicking "Add Friend" or "Follow"). Users can also input content into the community feed by creating a new post – this could be a text post (like *"Anyone going to the library this afternoon?"*), possibly with an image attachment. Additionally, users can comment on posts, which involves text input in a comment field.
- **Processing:** When a user sends a friend/follow request, the system records the relationship (pending or confirmed) between the two user accounts. If privacy settings allow, the system may then share certain information between friends (such as

showing a friend's avatar on a map when they are checked in nearby, or showing a list of a friend's favorite locations). For the community feed, the app stores each post in a feed database with timestamp and author info. Fetching the feed involves retrieving recent posts, along with any comments associated with them. The system orders posts typically by most recent (or possibly by popularity if that is a factor). Comments on a post are processed similarly by linking them to the post and storing them.

- **Outputs:** In the UI, the **Community Feed** section (often accessible via a "Community" tab) displays a scrolling list of posts. Each post shows content, author name, time, and a count of comments or likes if applicable. When a user adds a new post, it appears at the top of the feed for others to see. Comments appear underneath their respective posts (often via a tap to view comments). For friend features, outputs include a list of friends on the user's profile (or a count of followers/following), and possibly indicators on the map or elsewhere showing friend activity (e.g. "Your friend Alice is at Café X now"). If a user chooses not to engage with social features, these elements can remain hidden or inactive. Overall, this requirement outputs a more **collaborative experience**: users can share experiences and connect, turning QuietSpace into a community-driven platform rather than just a directory of places.

FR11: Customization and Settings

- **Introduction:** QuietSpace provides optional **customization features** to improve user comfort and personal preference. This includes visual theme options (such as light or dark mode) and other UI preferences, as well as privacy controls for the user's account. While not core to finding locations, these features enhance the user experience by allowing the app to adapt to user needs and preferences.
- **Inputs:** The user's selection in the settings menu. For theme switching, input might be a toggle switch between Light and Dark mode (or a list of theme choices). For other settings, inputs could include toggling notification preferences (on/off for various notifications), adjusting map display preferences (like map style or units), and setting privacy options (e.g. making the profile private or public). The user navigates to a Settings or Preferences screen to provide these inputs.
- **Processing:** When the user changes a setting, the app applies the change immediately if it's a local UI setting (e.g. switching to dark mode will trigger the app to load the dark color palette and re-render screens accordingly). It also persists the setting – for instance, saving the preference in the app's local storage or the user's profile record in the backend so that it remains in effect in future sessions or across devices. For privacy settings, processing may involve updating the user's profile flags on the server (e.g. a field indicating the profile is private) and the app logic will then respect those flags (for example, not showing that user in friend search results if private).

- **Outputs:** The immediate output is the modified appearance or behavior of the app. For example, if the user enables Dark Mode, the app's color scheme changes instantly to a dark background style. If the user turns off a certain notification, the output is less visible – it's essentially that the app will no longer output those notification messages. For privacy toggles, the output is more indirect: the user's profile will no longer be visible to others as specified (which the user perceives as a protection output). In all cases, a confirmation (such as a brief message or just the visible effect taking place) assures the user that their preference has been applied. These customization features ensure that QuietSpace is **user-friendly and adaptable** to individual needs.

3.2 Use Cases

The following use cases describe how users interact with the QuietSpace application to achieve specific goals. Each use case includes a brief description and a step-by-step scenario of the typical interaction. (Each use case is assumed to start with the user launching the app and logging in, unless otherwise noted.)

Use Case 1: Discover a Quiet Location (Map Browsing)

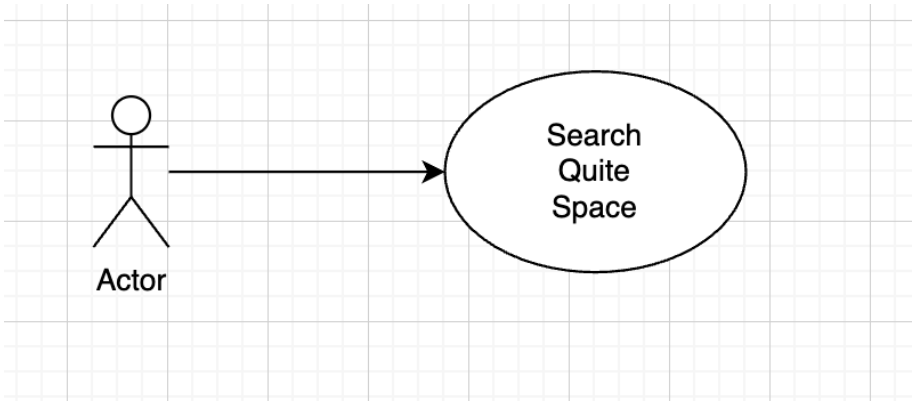


Diagram: *The user (actor) interacts with the QuietSpace app's map interface to find a suitable quiet location.*

Brief Description: This use case covers a user exploring the map to discover quiet study/work spaces near them. The user opens the map, views location pins, and selects a location to see details.

Initial Step-By-Step Description:

Precondition: The user has opened the app and has location services enabled. The app has fetched the user's current location.

1. **User** opens the QuietSpace app and navigates to the map screen (home screen by default).

2. The **System** displays an interactive map centered on the user's location, with pins representing nearby quiet locations.
3. **User** zooms and pans the map to browse different areas (or types a query in the search bar for specific places).
4. The **System** updates the map, loading more location markers as the view changes (and filtering results if the user used the search/filter).
5. **User** taps on a location pin that looks interesting.
6. The **System** opens a small info card showing key details (name, brief description, rating, etc.) for that location.
7. **User** taps the info card to see more details.
8. The **System** opens the detailed location page, showing full information (noise level, Wi-Fi quality, user reviews, etc.). The user can now decide to go to this location or go back to the map to explore others.
Postcondition: The user has identified a quiet location that meets their needs (and may proceed to navigate there in real life).

Use Case 2: Add a New Quiet Location

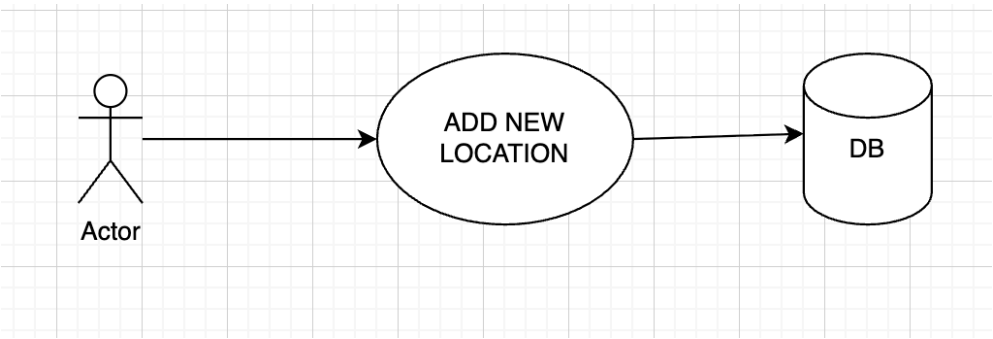


Diagram: *The user (actor) submits a new location through a form in the QuietSpace app.*

Brief Description: In this use case, a user contributes a new quiet space (for example, a café or study hall) to the QuietSpace community by filling out the required details.
Initial Step-By-Step Description:
Precondition: The user is logged in and on the map or community screen of the app. They have a new location in mind that is not yet in the system.
1. **User** clicks the "Add Location" button (often represented with a "+" or an "Add" icon) in the app.
2. The **System** presents a form for adding a new quiet location, which includes fields for name, address or map pin, and various location details (tags, features, etc.).
3. **User** enters the location name and either enters the address or drops a pin on

the map to specify the exact location. They fill in descriptive details: e.g., *"Quiet hours upstairs, good for groups, outlet at every table"*. They also select tags/options for noise level (e.g. quiet), Wi-Fi (fast/free), outlet availability (many), and any other prompts. Optionally, the user uploads a photo.
4. **User** submits the form by tapping "Save" or "Submit".
5. The **System** validates the input (ensures required fields are filled and the location isn't a duplicate).
6. The **System** saves the new location to the database and then updates the map view to include the new location's pin.
7. The **System** confirms success to the user (e.g., with a message "Location added successfully!").
8. **User** can now see their contributed location on the map (and so can other users of QuietSpace).
Postcondition: A new quiet location is now available in the QuietSpace app for all users, and the contributing user's profile is updated (they may gain points/badges for this contribution).

Use Case 3: Check In and Update Location Status

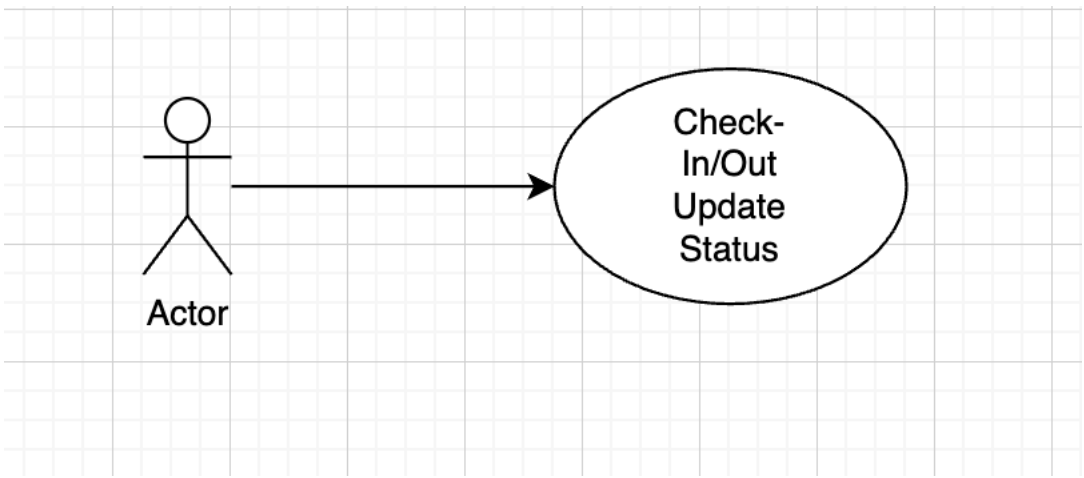


Diagram: *The user (actor) checks in at a quiet location, updating real-time information.*

Brief Description: This use case describes a user providing a real-time update for a location by checking in. By doing so, the user shares current conditions (like noise level and crowding) with the community.
Initial Step-By-Step Description:
Precondition: The user is at a known quiet location (physically present) and has the QuietSpace app open. The user is logged in.
1. **User** navigates to the detail page of the location they are currently in (for

example, by selecting it on the map or from their favorites).
2. **User** taps the “Check In” or “Update Status” button on that page.
3. The **System** presents a quick check-in interface, showing a set of fields or sliders for reporting conditions (Noise Level, Wi-Fi Quality, Seating Availability, etc.). It may auto-fill some fields (like current time, user’s name) behind the scenes.
4. **User** selects or inputs the current conditions. For instance, they set Noise Level = *Quiet*, Wi-Fi = *Strong*, Seating = *Some available*, and optionally type a short comment (“A bit cold near the door, but very quiet upstairs.”).
5. **User** submits the check-in update.
6. The **System** records this update. It updates the location’s live data (e.g., marking noise level as quiet as of now, incrementing a “recent check-ins” count, etc.).
7. The **System** might display the user’s contribution immediately on the location page (e.g., showing the new average or latest status). If other users are viewing this location, they will see the new update reflected.
8. **User** sees a confirmation that their check-in is successful (perhaps a toast message like “Thanks for updating! Your check-in is now live.”). They might also see their contribution listed (for example, their comment appears with a timestamp and their name in a “Recent Updates” section for that place).
Postcondition: The location’s details are refreshed with the latest user-supplied information, and the community benefits from up-to-date knowledge (the contributing user potentially earns reward points for this action as well).

Use Case 4: Register and Set Up Profile

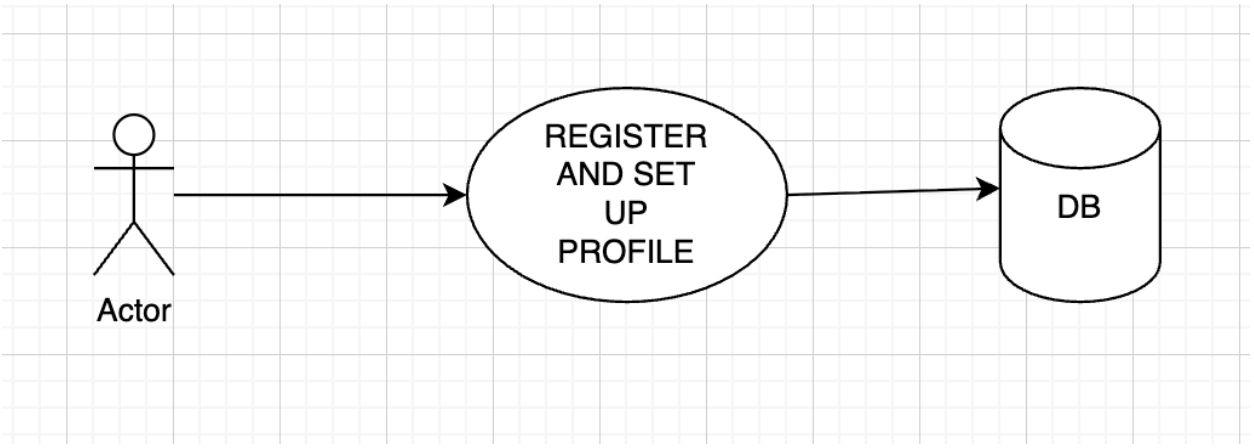


Diagram: *The user (actor) creates a new QuietSpace account and sets up their profile.*

Brief Description: This use case covers a new user signing up for QuietSpace and

customizing their profile. It ensures the user can securely register and then add personal details to their profile.

Initial Step-By-Step Description:

Precondition: The user has installed the QuietSpace app and launches it for the first time (or has logged out).

1. **User** chooses the “Sign Up” option on the welcome screen.
2. The **System** displays the account registration form. This includes fields like Name, Email, and Password, and may offer “Sign up with Google/Apple” buttons.
3. **User** fills in the form with a valid email address, a password, and their name. (If using Google/Apple, they authorize via the external service, and the system returns with the needed info.)
4. **User** submits the registration form.
5. The **System** validates the inputs (checks email format, password strength, etc.). If everything is valid, the system creates a new user account.
6. The **System** logs the user into the app immediately after successful registration.
7. The **System** then prompts the user to set up their profile (this could be an optional onboarding step). For example, it may ask the user to upload a profile picture and add a short bio, or to configure initial preferences (like enabling location permission, selecting interests).
8. **User** uploads a profile photo and writes a one-line bio (e.g., “GBC student who loves quiet cafés.”). They save their profile info.
9. The **System** saves these profile details to the user’s account.
10. The **System** takes the user to the main app (map view) now that setup is complete, possibly with a welcome message or tutorial tips.

Postcondition: The user is now registered in QuietSpace with a personalized profile. They can fully participate (e.g., add locations, reviews, favorites) and their profile info will be visible according to default privacy settings.

(Alternate Flow: If the email is already taken or the password is too weak, the system will show an error and prompt the user to correct the input. Registration only completes after valid credentials are provided.*)*

Use Case 5: Save a Location to Favorites

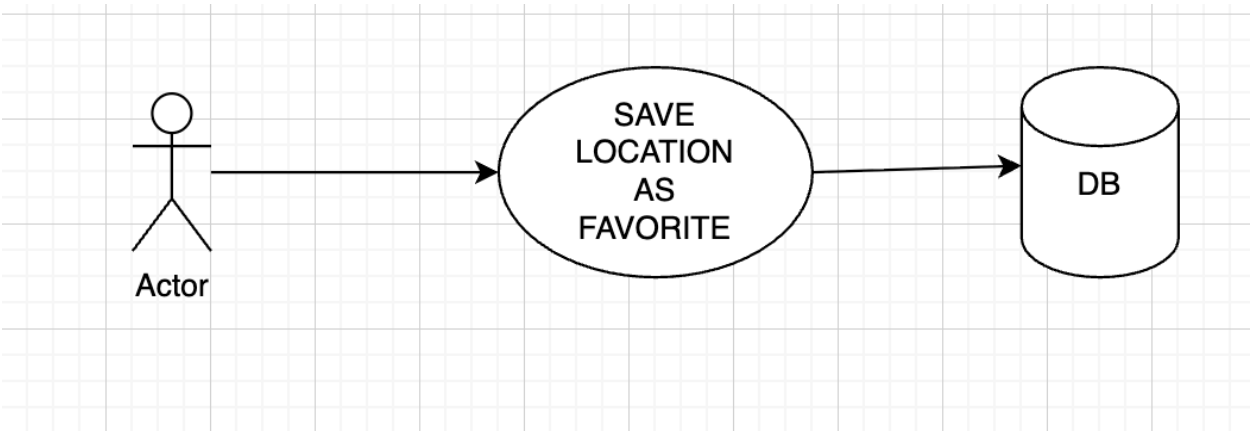


Diagram: *The user (actor) marks a quiet location as a favorite in the app.*

Brief Description: In this use case, a user bookmarks a quiet location for easy access later. The location will be saved to their favorites list.

Initial Step-By-Step Description:

Precondition: The user is logged in and viewing details of a quiet location (having found it via the map or search).

1. **User** decides they really like a particular location's details and wants to save it. They tap the "Favorite" icon (heart/star) on the location's detail page.
2. The **System** registers this action and immediately highlights the icon (for instance, filling in the heart icon) to indicate the location is now favorited.
3. The **System** adds this location to the user's list of favorite locations in the backend.
4. **User** can now access this saved place by going to the "Favorites" screen. They navigate to their Favorites list from the menu.
5. The **System** displays the user's Favorites screen, which now includes the newly added location among any others. Each entry shows the location name and perhaps a thumbnail or key info.
6. Later, if the **User** wants to remove the location from favorites, they simply tap the filled heart icon again on that detail page (or maybe swipe/remove it from the favorites list). The **System** would then remove it from their saved list and update the icon to unfilled.

Postcondition: The user has a personalized favorites list that persists across sessions. They can quickly refer to these saved spots without searching through the map every time. This improves user convenience especially for frequently visited places.

Use Case 6: Review and Rate a Quiet Location

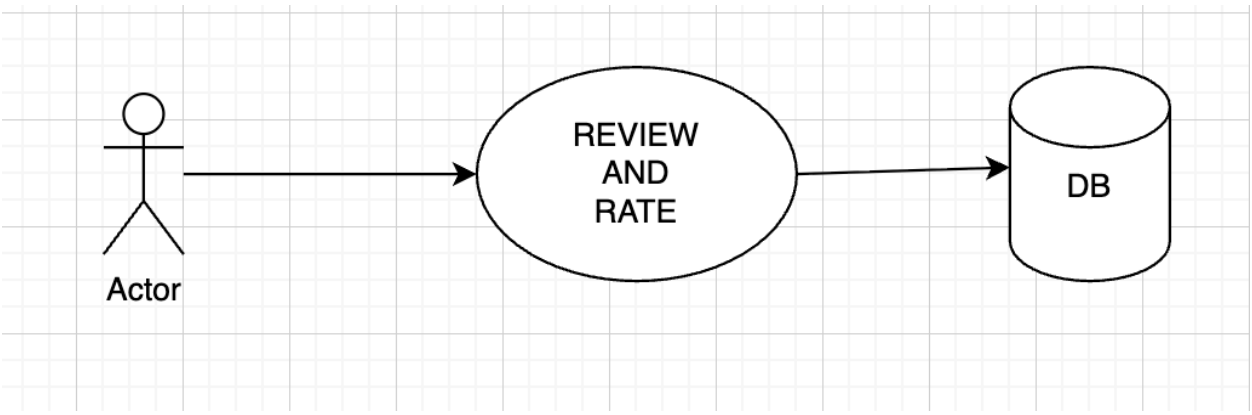


Diagram: *The user (actor) submits a review for a location; other users (actors) read reviews on that location.*

Brief Description: This use case involves a user providing feedback on a location by writing a review and giving a rating, and it covers how that information is then available to others.

Initial Step-By-Step Description:

Precondition: The user is logged in and has visited a location (or has sufficient knowledge about it) and is viewing that location's page in the app.

1. **User** navigates to the location's detail page (which shows existing reviews and an average rating).
2. **User** taps on "Write a Review" or a similar call-to-action.
3. The **System** displays a review submission form: a star rating control and a text field for comments.
4. **User** selects a rating (e.g. 4 out of 5 stars) and types their comment in the text field (e.g., "Great quiet atmosphere on weekday mornings, but gets noisy during lunch rush.").
5. **User** submits the review.
6. The **System** validates the input (ensuring a rating is selected and that the comment isn't flagrantly violating any format rules, for instance).
7. The **System** saves the review and rating to the database, associated with that user and location, and recalculates the location's overall rating (if using an average).
8. The **System** updates the UI: the new review now appears in the reviews list for that location, and the average rating is updated (if affected by the new input). For example, other users might now see that the location has 11 reviews instead of 10, and maybe the average moved from 4.6 to 4.5 stars.
9. **User** sees a confirmation message or simply sees their review posted. Their

profile's review count increases by one.
Postcondition: The location's information is richer – it now includes the user's firsthand feedback. Other users can read this new review when they view the location, helping them make informed decisions. The contributing user's reputation in the app grows via the added review (which could contribute to points/badges).

Use Case 7: Engage with the Community Feed

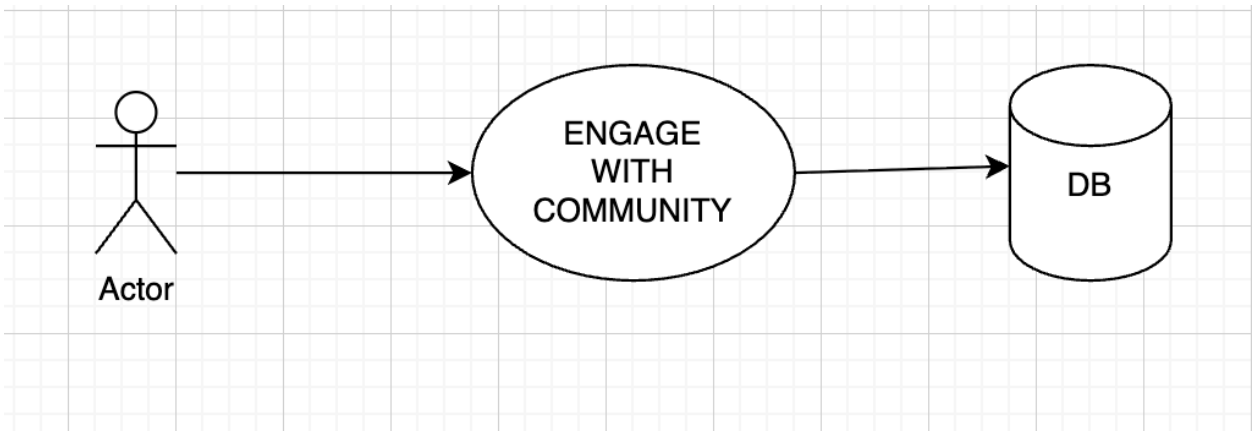


Diagram: *The user (actor) posts a message to the community feed and may interact with others' posts.*

Brief Description: This use case illustrates a user participating in the QuietSpace community by creating a post (for example, to organize a group study or ask for recommendations) and optionally commenting on others' posts.
Initial Step-By-Step Description:
Precondition: The user is logged in and has navigated to the Community section of the app.
1. **User** taps on the "Community" tab in the navigation bar.
2. The **System** displays the community feed – a list of recent posts from users (with their names, post time, and content preview).
3. **User** reads through some posts. Suppose they want to make their own post asking a question. They tap the "New Post" button (e.g. an icon or text field that says "Share something…").
4. The **System** opens a text input screen for a new post.
5. **User** types a message, for example: "Looking for a quiet place near campus to study this evening. Any recommendations?" They hit "Post."
6. The **System** publishes the post to the community feed. This includes stamping it with the current time and the user's identity.
7. The **System** refreshes the feed; the new post appears at the top of the list for the

user and will be visible to others as they refresh or scroll.
8. Later, another **User (Friend)** sees the post and taps on it to answer. They type a comment in response ("Try the Maple Library, it's open late and usually empty on Fridays.") and submit it.
9. The **System** adds that comment under the original post. The original poster (from step 1) might get a notification that someone replied.
10. **Original User** can check back on their post, see the comment, and perhaps reply or mark it as helpful.
Postcondition: Users have engaged socially: the original user's question was addressed by the community. The feed now contains this interaction for others to see and learn from (other users might also chime in or benefit from the answers). This use case fosters a collaborative community environment beyond just location listings.

Use Case 8: Utilize the Pomodoro Timer (Optional)

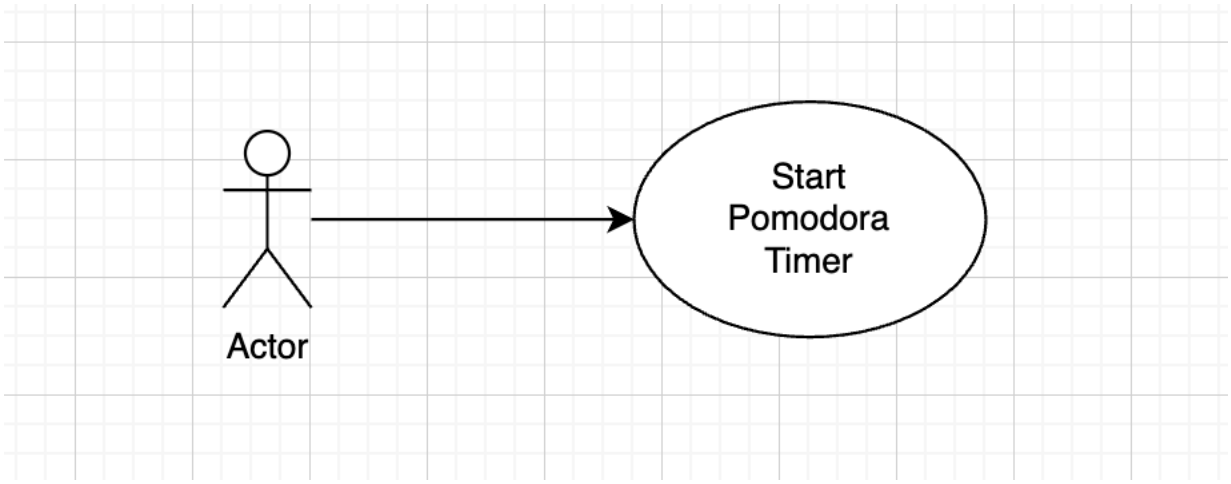


Diagram: *The user (actor) uses the built-in Pomodoro timer while at a quiet location.*

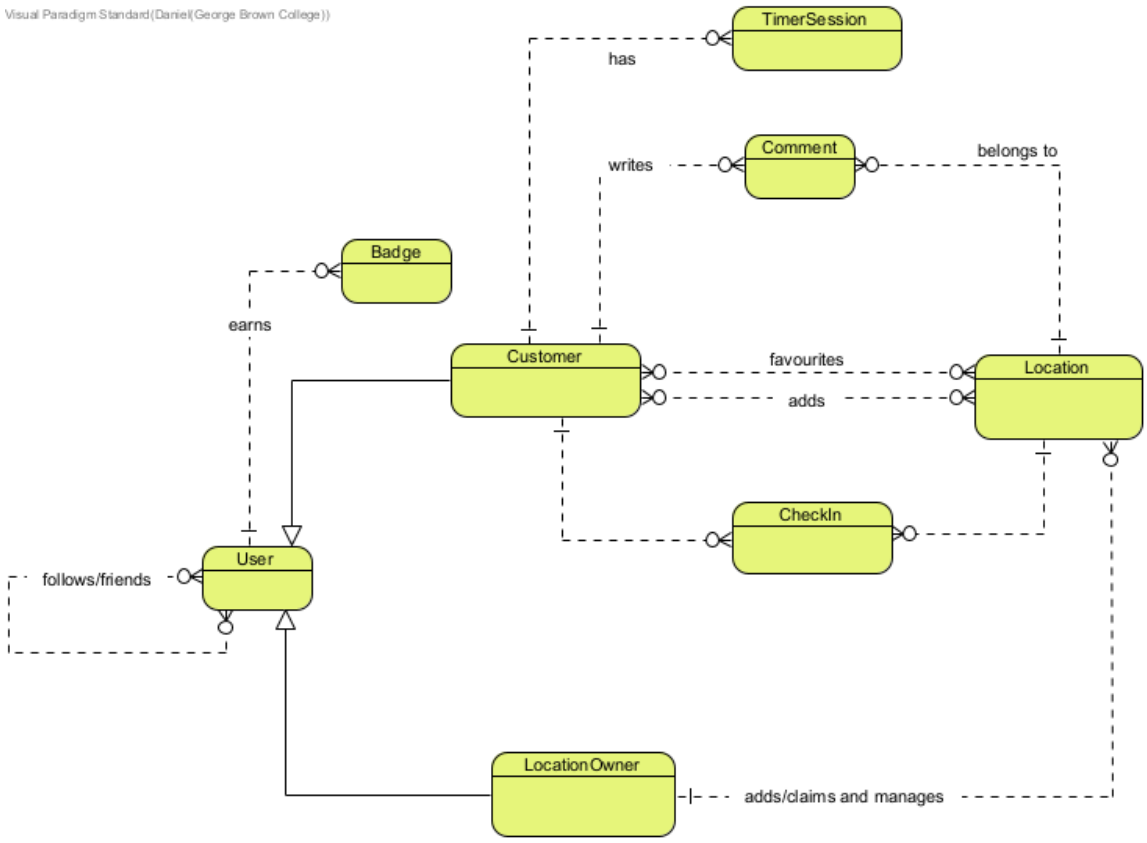
Brief Description: This use case describes how a user starts and uses the Pomodoro focus timer within QuietSpace to manage their study session.
Initial Step-By-Step Description:
Precondition: The user is logged in and possibly on the location detail page or a dedicated Productivity section of the app. The user is ready to begin a focus session.
1. **User** navigates to the Pomodoro timer feature. (This might be accessible via a button on a location page like "Start Focus Session" or through a menu labeled "Timer".)
2. The **System** displays the Pomodoro timer interface, showing default work/break durations (for example, 25:00 for work and 5:00 for break) and controls to start,

pause, or adjust settings.
3. **User** optionally adjusts the timer settings (for instance, sets work period to 30 minutes and break to 5 minutes, or decides to stick with the default).
4. **User** taps the “Start” button.
5. The **System** begins counting down the work timer. The timer runs visibly in the app; the user might minimize it or let it run while they study.
6. When the work interval ends, the **System** alerts the user – perhaps by a gentle notification sound or pop-up that says “Time for a break!”. It also automatically starts the break timer countdown.
7. **User** takes a short break (possibly indicated in the app). When the break ends, the **System** notifies the user to resume work. This cycle may repeat for a set number of rounds or until the user stops the timer.
8. **User** can pause or stop the timer at any point (for example, if they finish studying early or need to interrupt the session). They tap “Stop.”
9. The **System** stops the timer and records the session if tracking is needed. It might display a brief summary (like “Focus session completed: 25 minutes focus, 5 minutes break”).

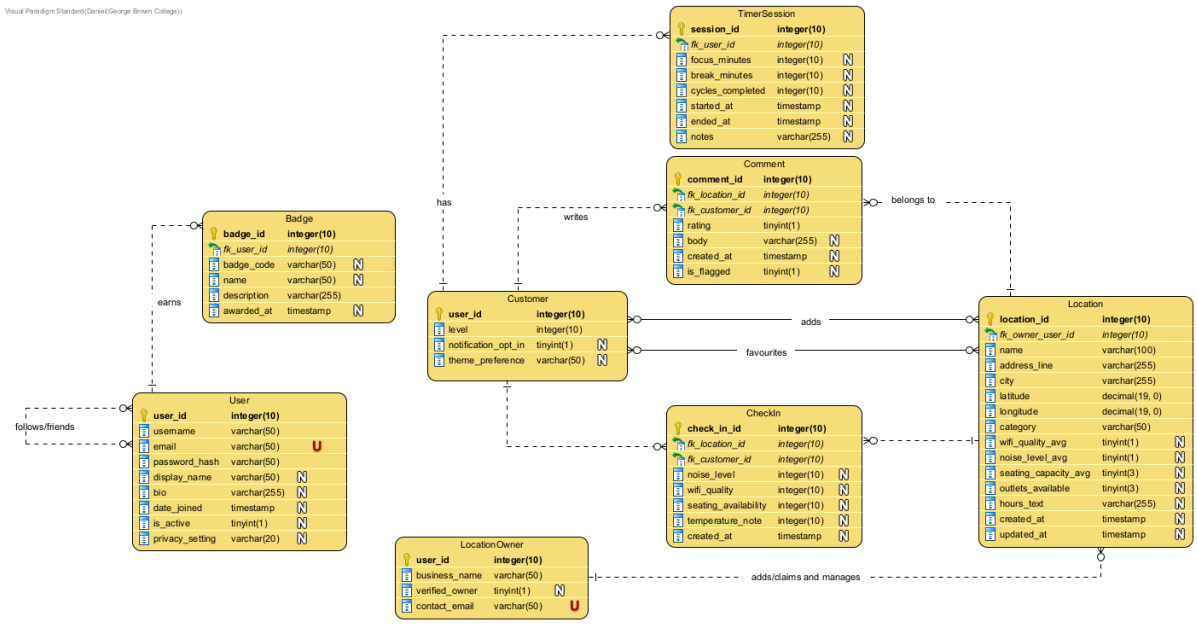
Postcondition: The user successfully used the timer to structure their study time. They remained focused during the work interval and took breaks as prompted. The timer helped manage their session within the QuietSpace app, demonstrating the app’s integrated productivity support.

3.3 Data Modelling and Analysis

3.3.1 Conceptual Model Diagram

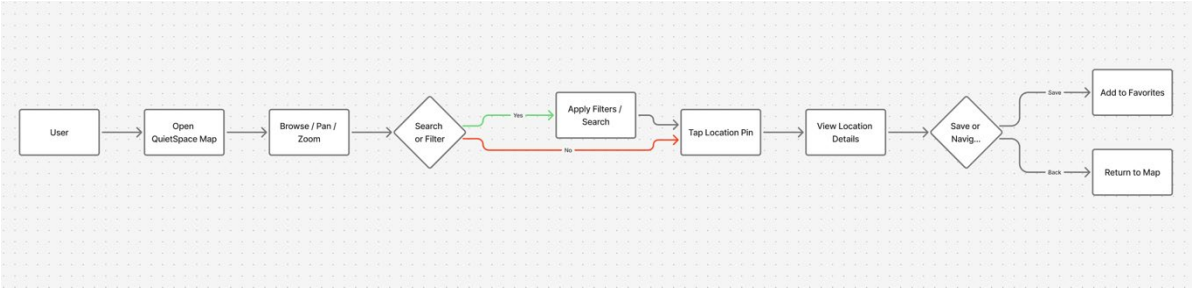


3.3.2 Normalized Data Model - Logical Model ERD

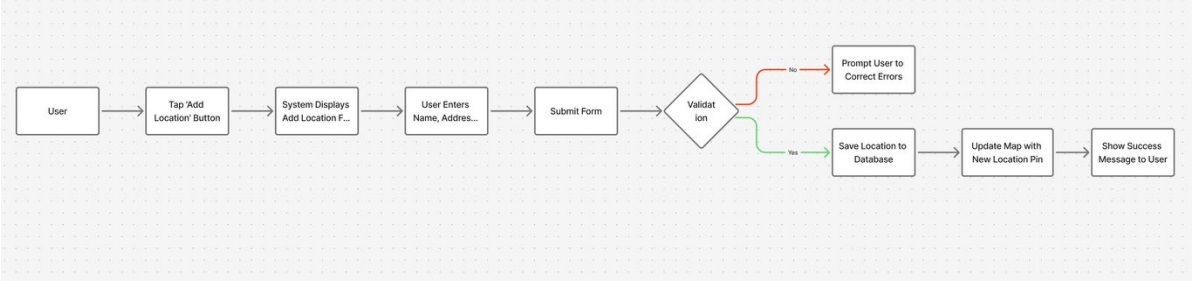


3.3.2 Activity Diagrams

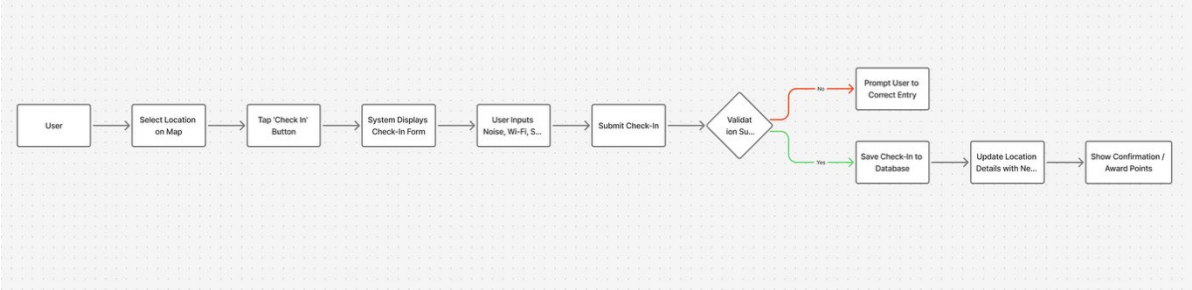
Discover Location / Search



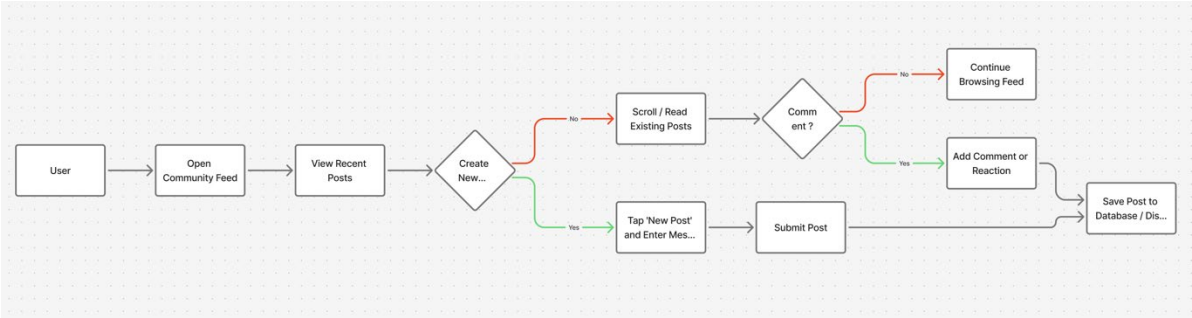
Add New Location



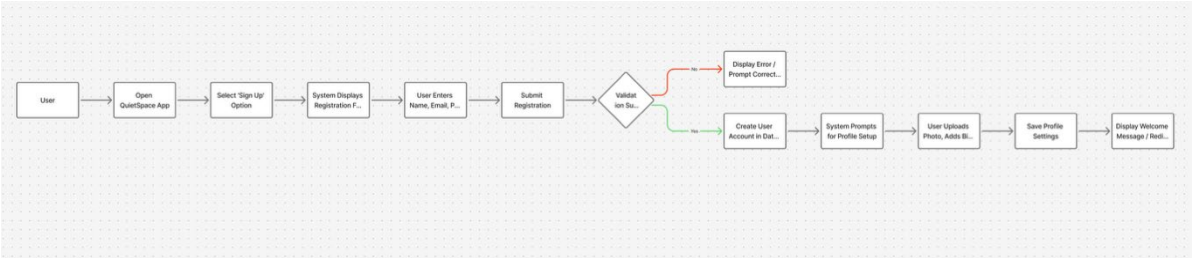
Check In Location



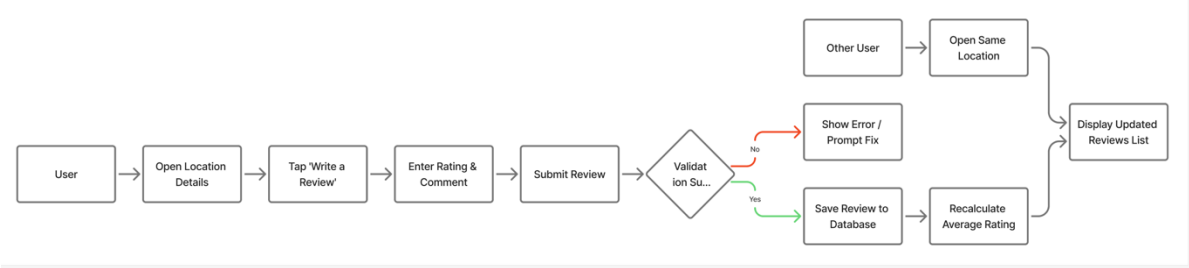
Engage with Community Feed



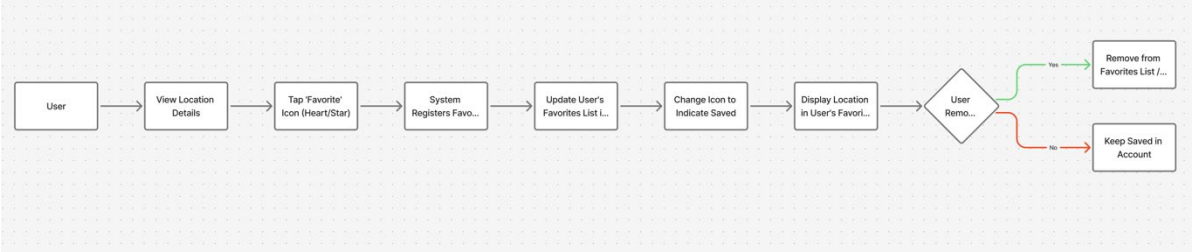
Create / Set Up an Account



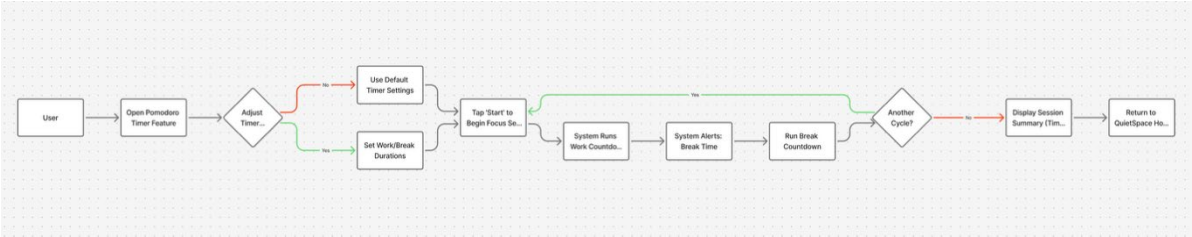
Submit / Read Review



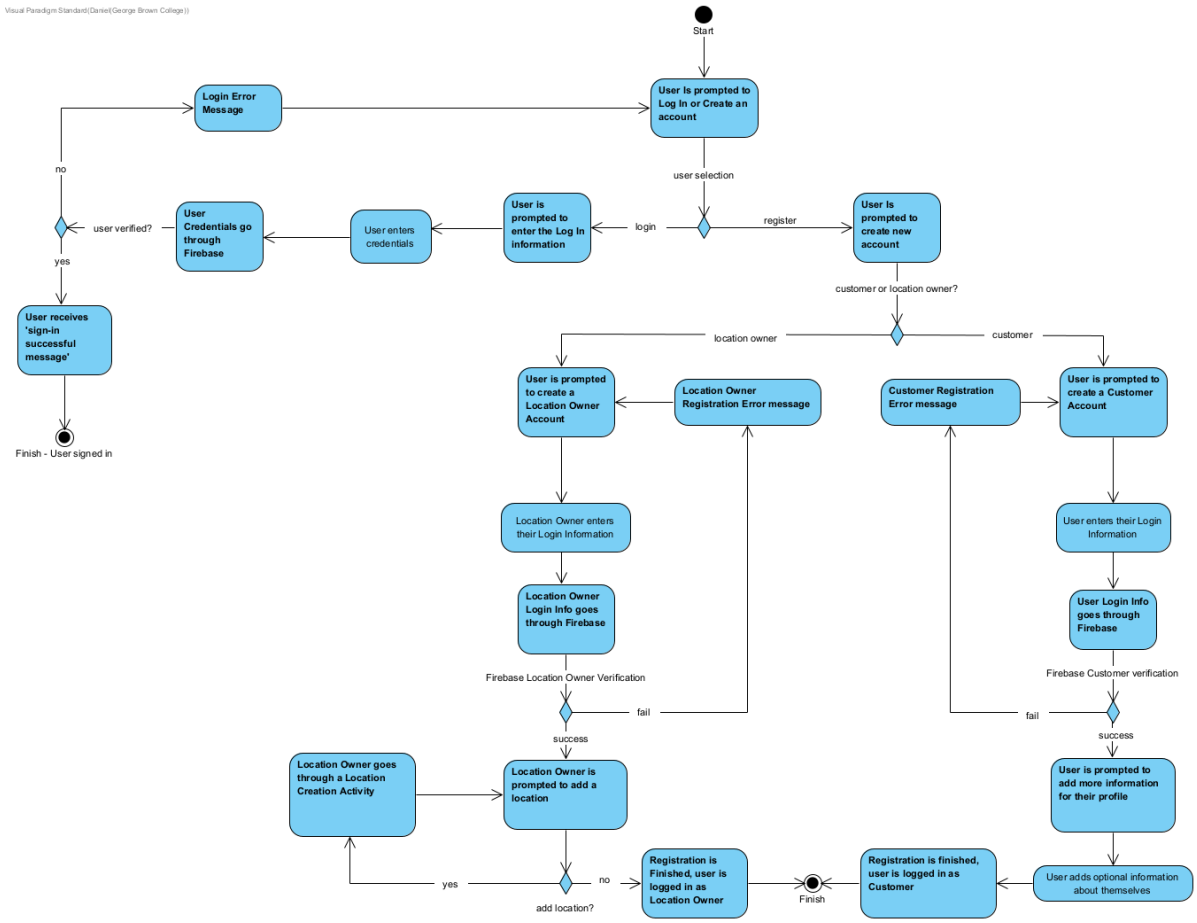
Mark Favorite



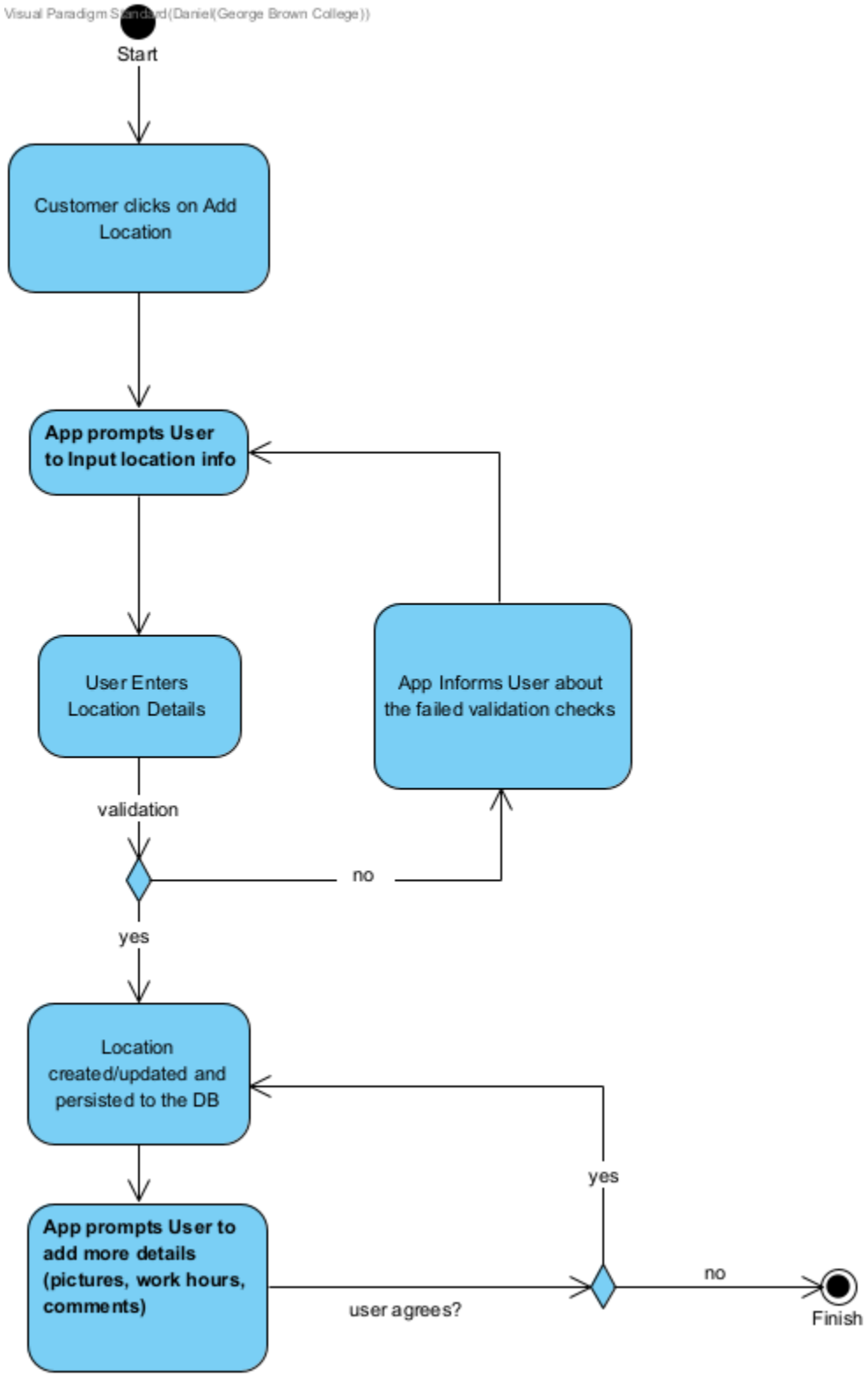
Use Pomodoro Timer



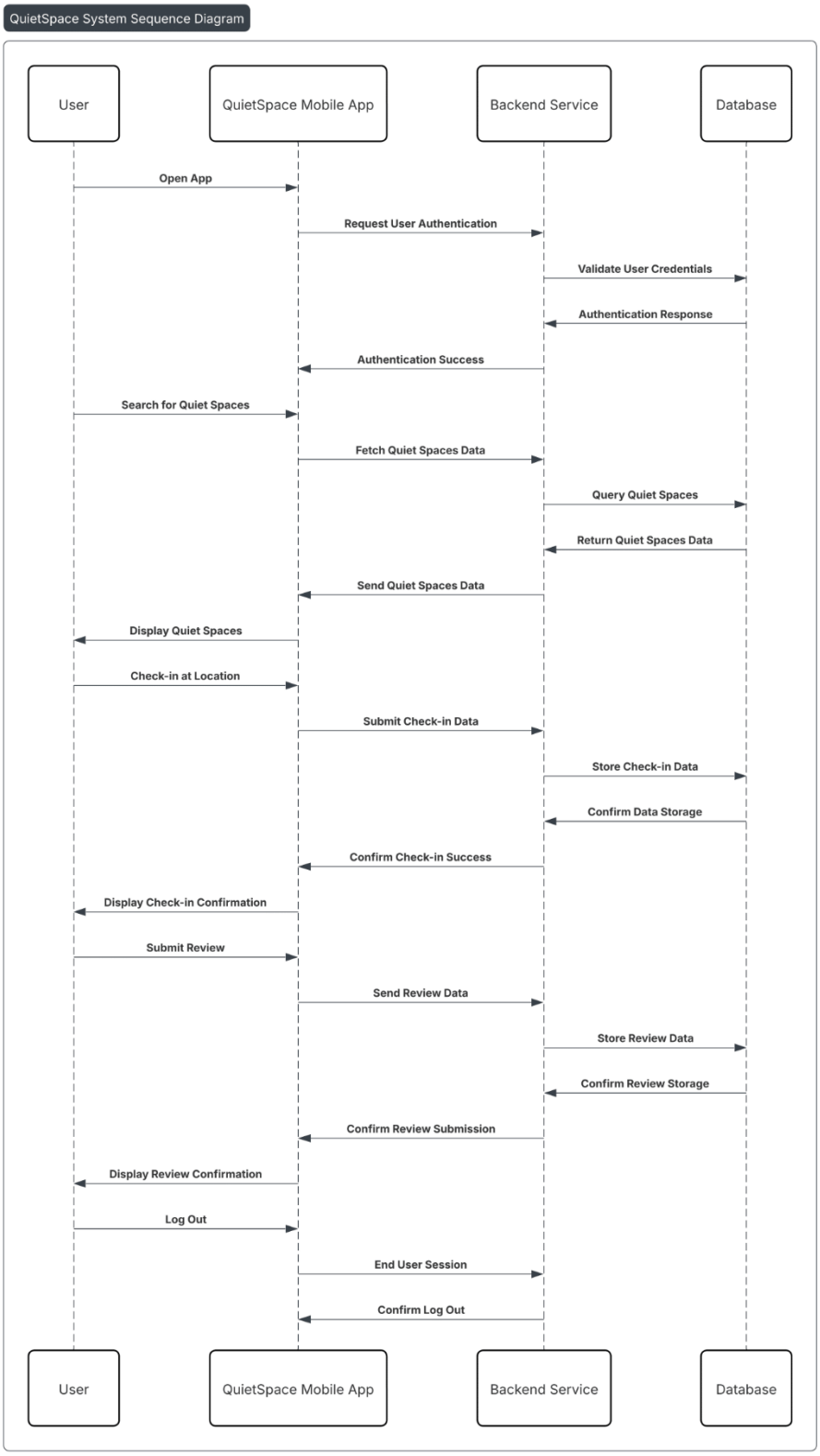
User Registration and Login Activity



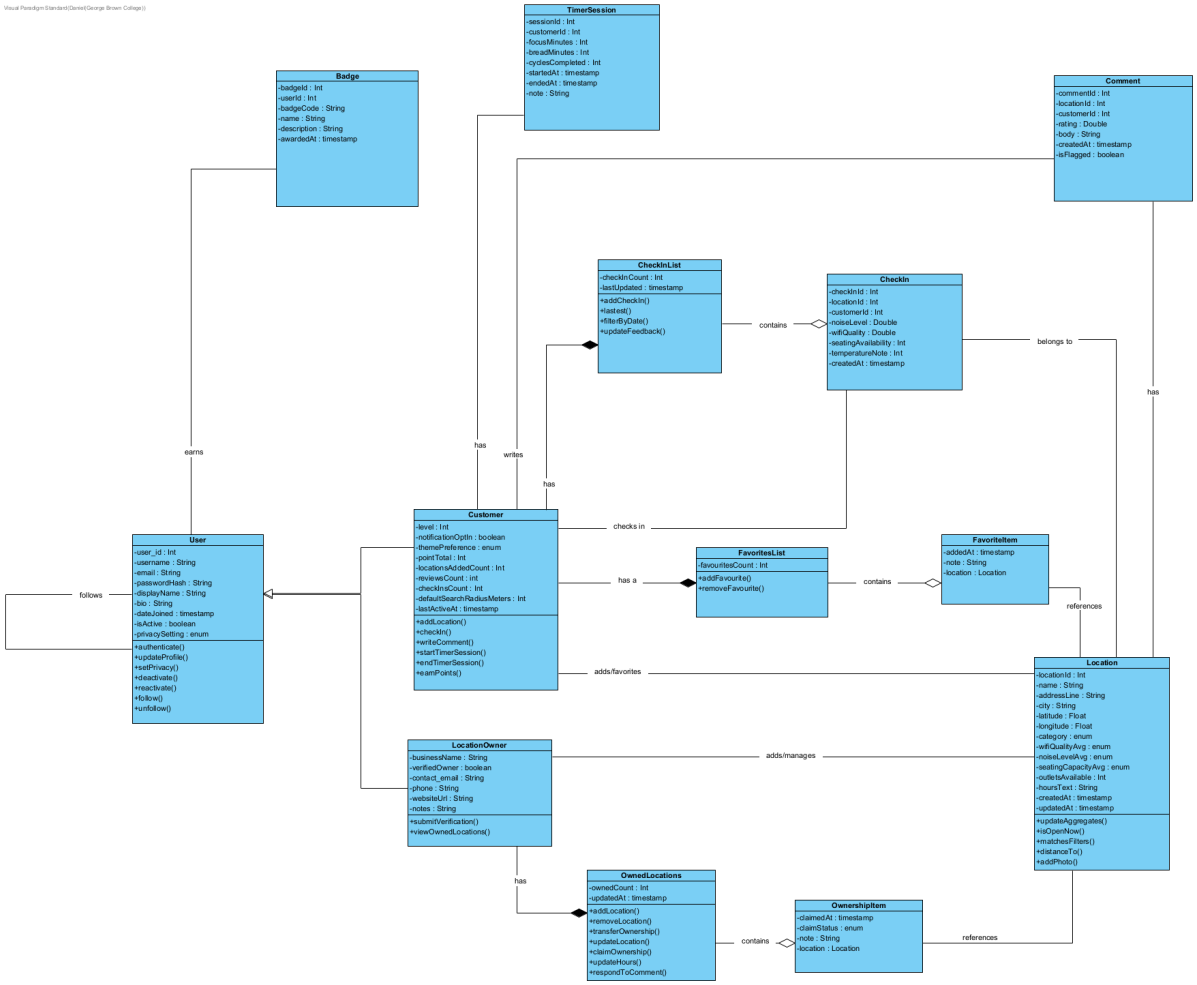
Add Location Activity



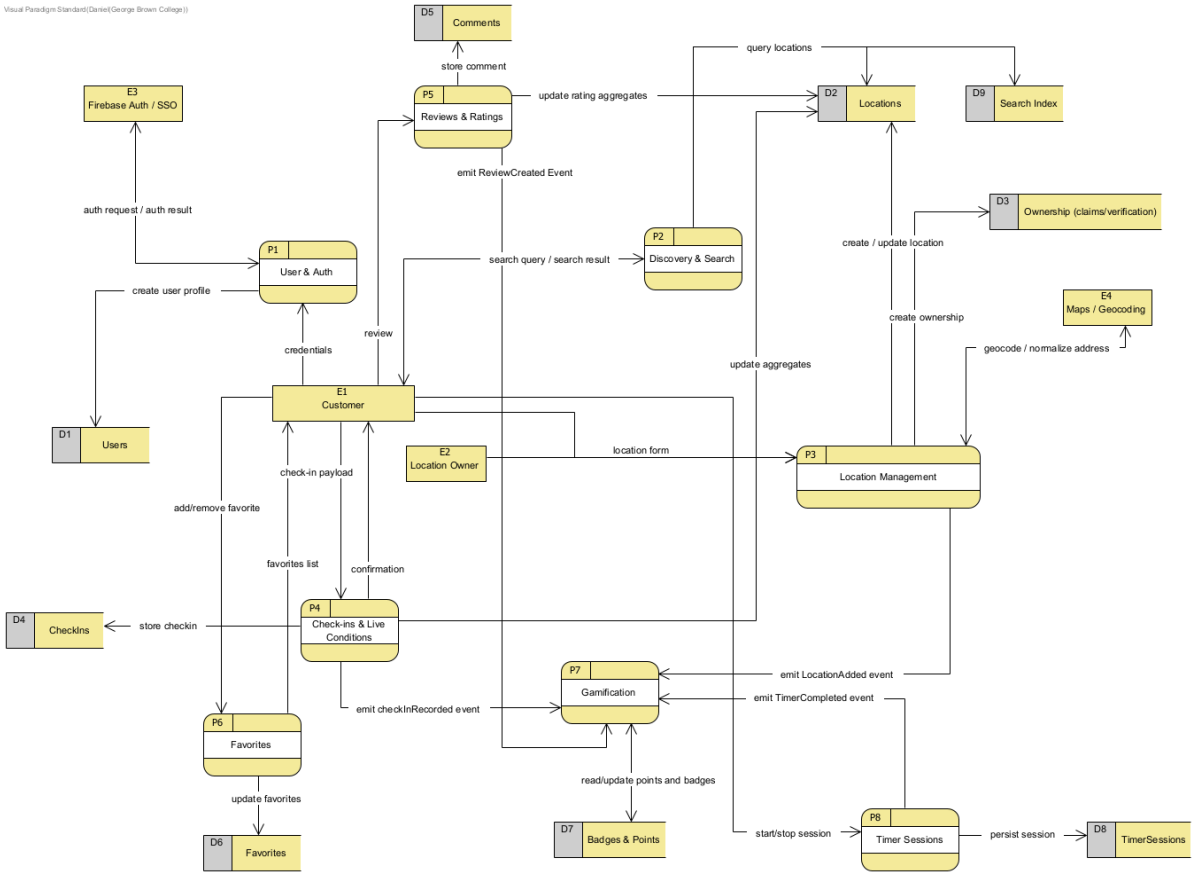
3.3.3 System Sequence Diagram



3.3.4 UML Diagram



3.4 Process Modelling – Data Flow Diagram



4.0 NON-FUNCTIONAL REQUIREMENT

Non functional requirements constrain how the system performs its functions. They are measurable and validated at a system wide level. The attributes below reflect the project plan and expected product behavior.

4.1 Performance

- The app will start within 3 seconds on mid-range Android devices.
- The map becomes interactive within 1.5 seconds after initial data load.
- 95% of check-in updates should be persisted and visible to other users within 2 seconds.
- 95% of push notifications for relevant location updates shall be delivered to devices within 10 seconds of the triggering event.
- The backend shall handle at least 15 concurrent active users per geographic region without degraded response times beyond the targets above.

4.2 Reliability and Availability

- System availability for core user-facing functions (map browsing, check-in, authentication) shall be 99.5% measured monthly.
- Total unplanned downtime should not exceed 30 minutes per month.
- If a dependent service (for example external geocoding) is unavailable, the app shall continue to allow read-only map browsing and keep cached favorites with clear indicators.
- All user-submitted location records and check-ins shall be stored with replication such that no single hardware failure can cause permanent data loss.

4.3 Security and Privacy

- Authentication uses trusted providers. Tokens are stored in platform secure storage. Role based controls restrict admin functions.

- Privacy by default. Social presence and location sharing are opt in. Users can request deletion and removal or anonymization of personal data within 30 days.
- Role-based access control shall be implemented for administrative and moderation interfaces.

4.4 Maintainability and Scalability

- Production deployments shall occur with automated CI/CD pipelines and have an automated rollback capability.
- New features shall include unit tests covering at least 80% of new code paths and automated integration tests for core flows.

4.5 Portability and Compatibility

- Mobile clients shall support Android API level 23 and above and iOS 14 and above on initial release.
- The mobile app shall support limited offline use: cached favorites and most recent map viewport data shall be available offline, with check-ins queued and synced when connectivity is restored.
- Basic location viewing and favorites management shall be accessible via a responsive web view for support and troubleshooting, with response times within 3 seconds for cached content.

4.6 Usability and Accessibility

- Basic location viewing and favorites management shall be accessible via a responsive web view for support and troubleshooting, with response times within 3 seconds for cached content.
- A first time user can search and read key amenities for a location within 60 seconds of install.
- User interface shall meet WCAG 2.1 AA standards for color contrast, scalable text, and screen reader navigability for primary screens.

5.0 LOGICAL DATABASE REQUIREMENT

5.1 Data Formats

4 All location records shall include structured fields: name, address, coordinates, noise level, Wi-Fi quality, seating availability, outlet access, timestamp, and contributor ID.
5 User profiles shall include: unique user ID, authentication method, preferences, contribution history, badge status, and privacy settings.
6 Check-in reports shall be stored as time-stamped entries linked to both user and location IDs.
7 Ratings and reviews shall include numeric scores, optional text.

5.2 Storage Capabilities

- It shall handle at least 100 location records and 500 check-in entries with horizontal scalability.
- Storage must support real-time writes and reads with low latency for mobile responsiveness.
- Historical data (e.g., past check-ins, badge history) shall be retained for analytics and gamification features.

5.3 Data Retention

- User-generated content (check-ins, reviews) shall be retained for a minimum of 12 months unless deleted by the user.
- Deleted user accounts shall trigger anonymization or removal of personally identifiable data within 7 days.
- System logs and audit trails shall be retained for at least 1 year for compliance and debugging.

5.4 Data Integrity

- All records shall include timestamps and contributor identifiers to ensure traceability.
- Duplicate location submissions shall be detected and merged using fuzzy matching and geolocation proximity.
- Ratings and reviews shall be validated to prevent spam or manipulation (e.g., one review per user per location per day).
- Referential integrity shall be enforced between users, locations, check-ins, and badges.

5.5 Security and Access

- Sensitive data (e.g., user credentials, location coordinates) shall be encrypted at rest.
- Role-based access control shall restrict administrative and moderation actions.
- All write operations shall be authenticated and logged.

5.6 Backup and Recovery

- Daily backups shall be performed with retention of at least 30 days.

6.0 OTHER REQUIREMENTS

Legal and compliance: explicit consent for location use and social visibility. In app privacy policy and terms are provided.

Backup and recovery: daily encrypted database backups with quarterly restore tests.

7.0 APPROVAL

The signatures below indicate their approval of the contents of this document.

Project Role	Name	Signature	Date
Team Leader	Daniil Orlov	Daniil Orlov	November 10, 2025
Quality Assurance	Gozde Baran	Gozde Baran	November 10, 2025
Product Owner	Ibrahim Yondem	Ibrahim Yondem	November 10, 2025
UI/UX	Baris Isci	Baris Isci	November 10, 2025
Developer	Nguyen Minh Triet Luu	Nguyen Minh Triet Luu	November 10, 2025